

Transformers

"Attention is all you need"



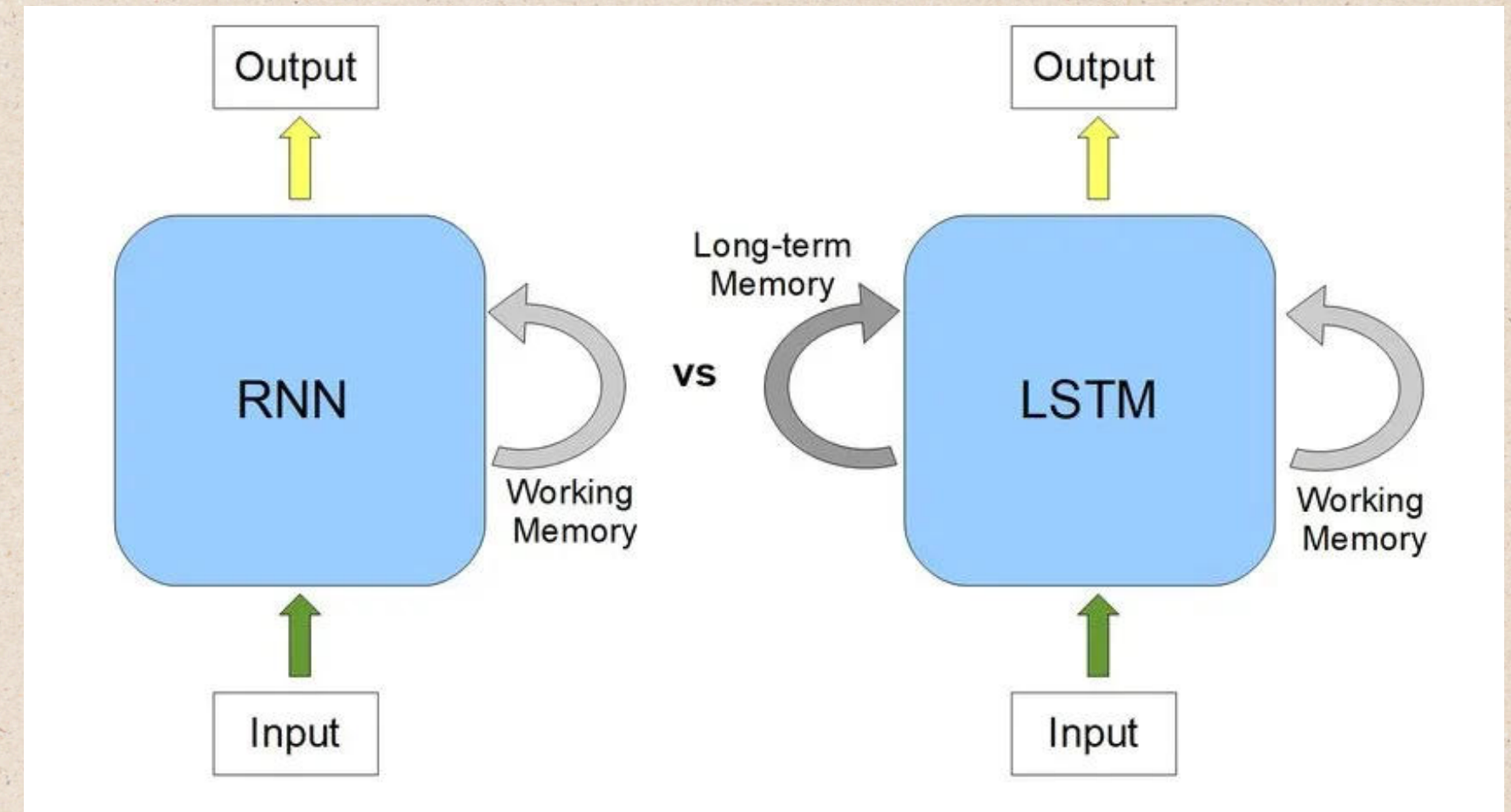
How did we reach here?

RNN

- Sequential
- Poor performance on long sequences

LSTM

- Sequential
- memory cells to store long information
- Better at capturing long-range dependencies
- But Struggles to focus on the most relevant parts of a sequence



Transformer

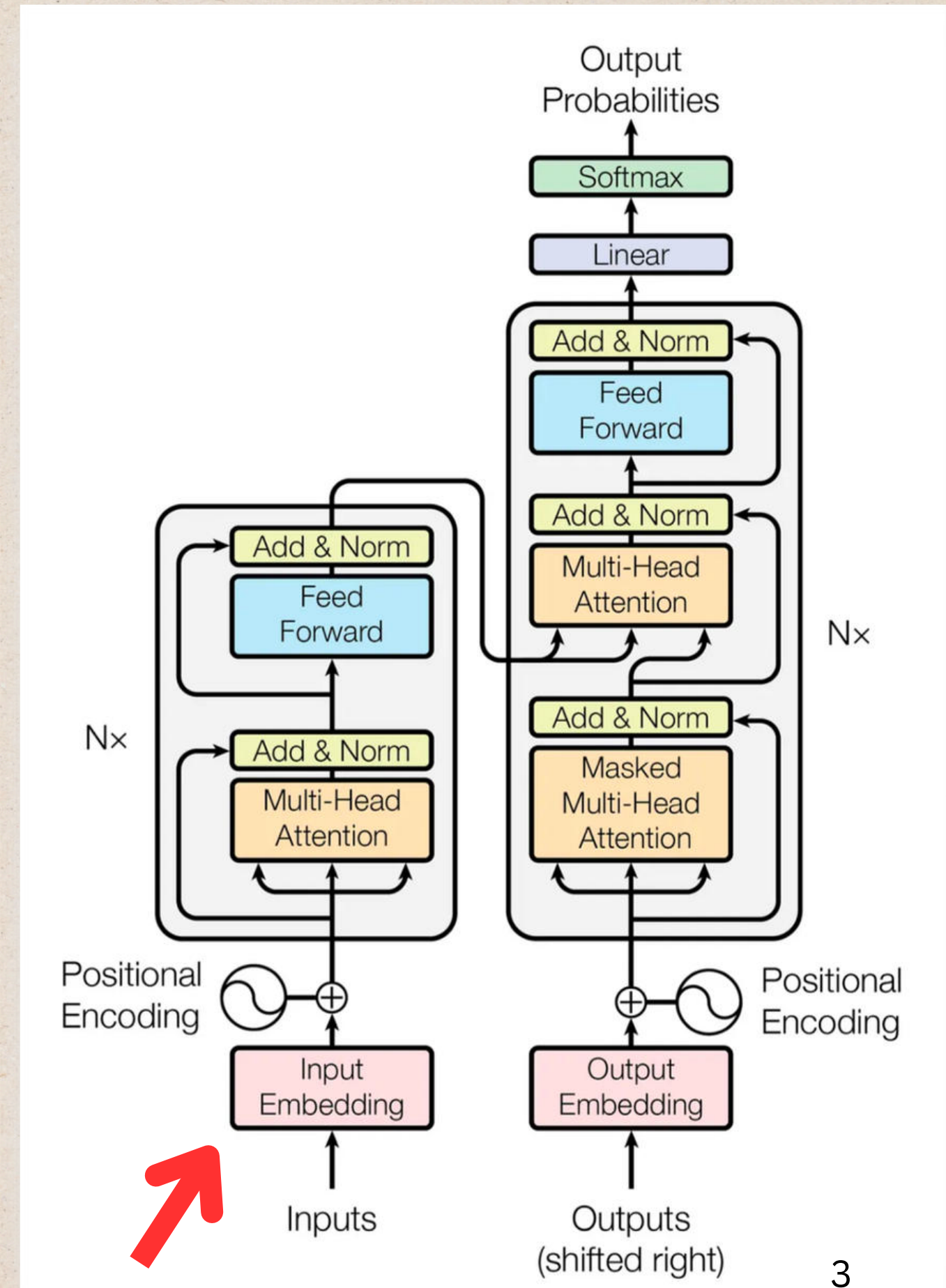
- "Attention is all you need" Published in 2017
- Parallel model but tasks are sequential
- self-attention to directly relate all words

Input Embedding

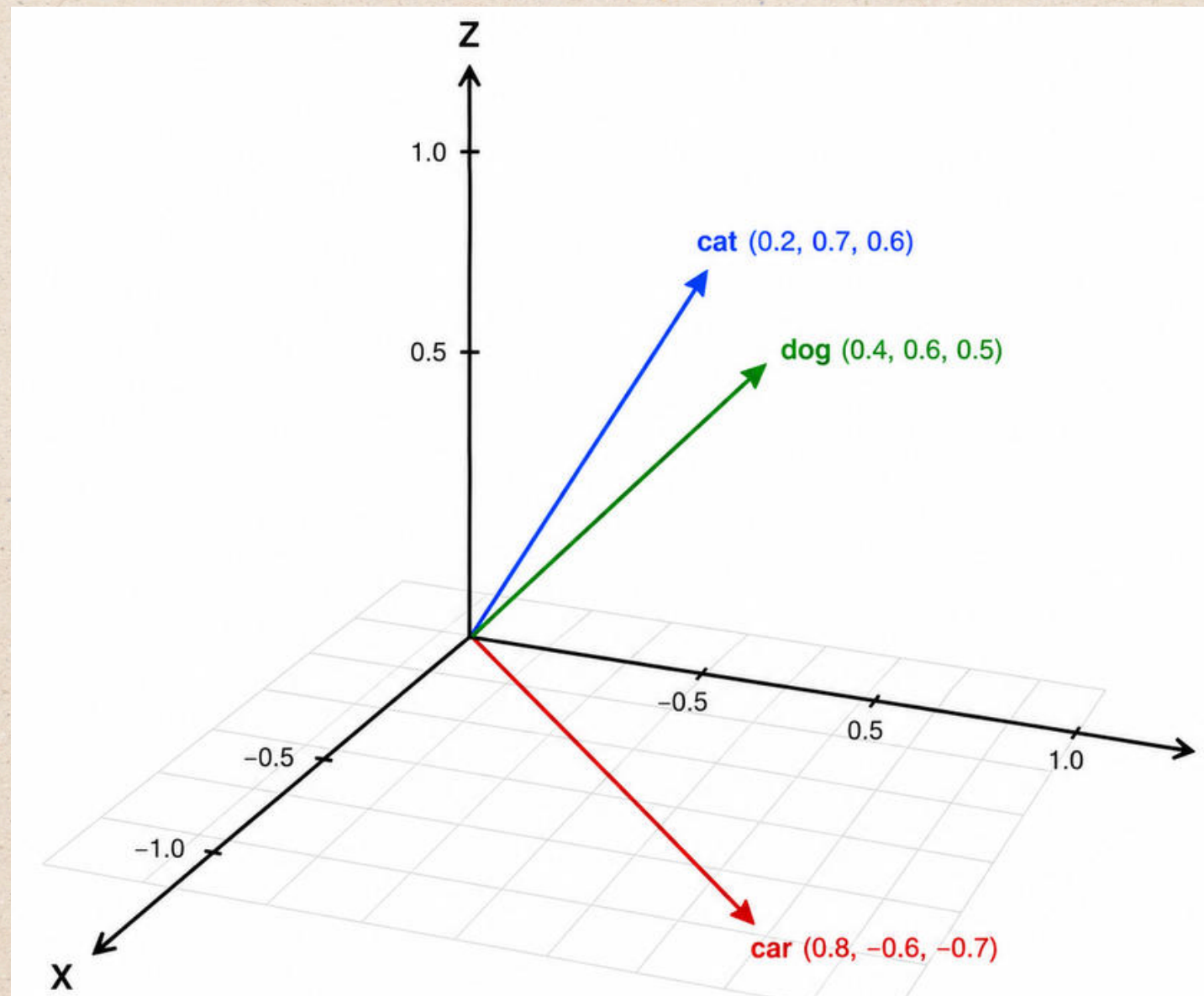
cat→[0.2, 0.1, -0.5] } Similar
dog→[0.2, 0.1, 0.4] }
car→[0.9, -0.2, 0.6] } Different

Embedding Vectors

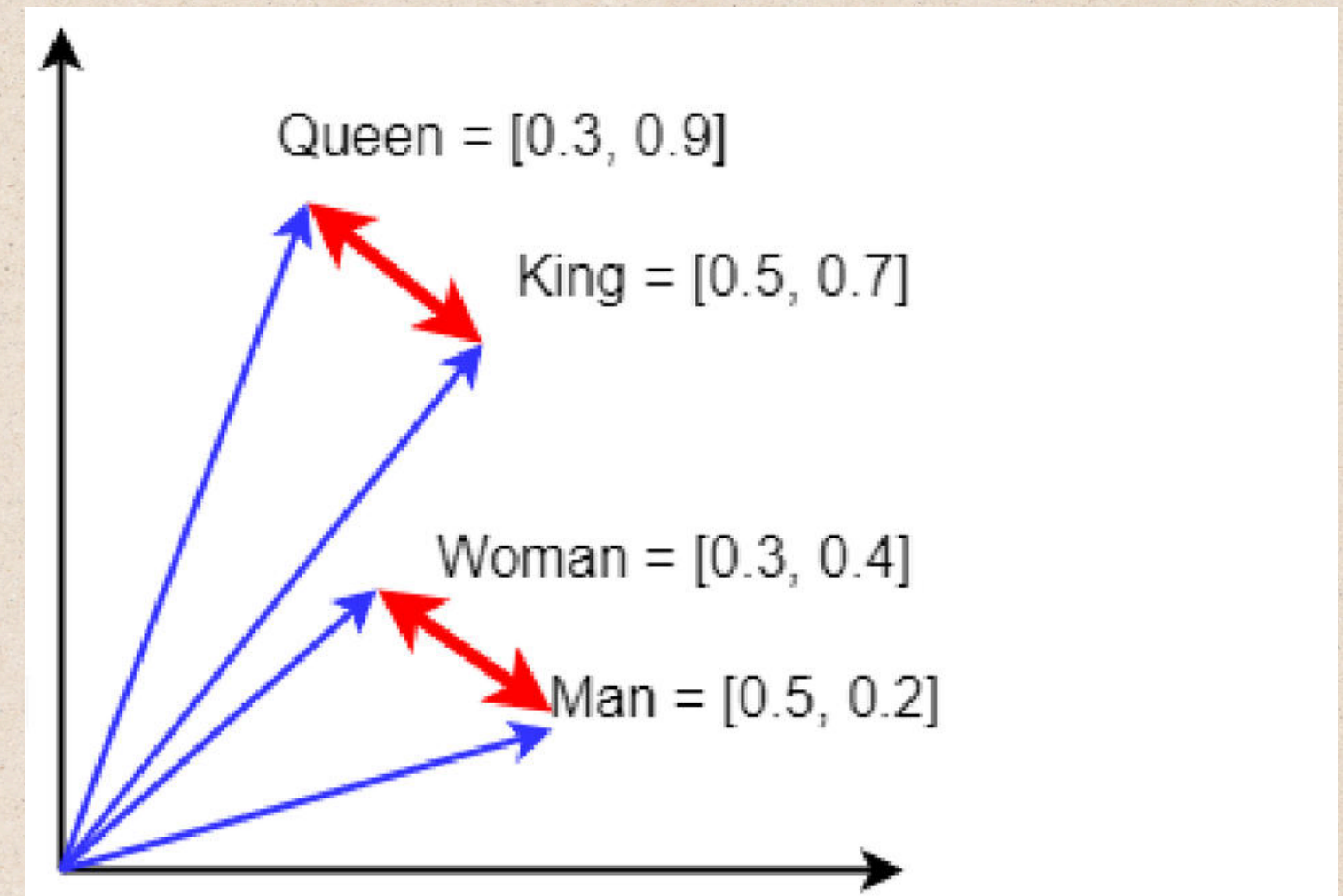
- Initially Random
- Embedding Vectors are also weights, so they get updated



Input Embedding



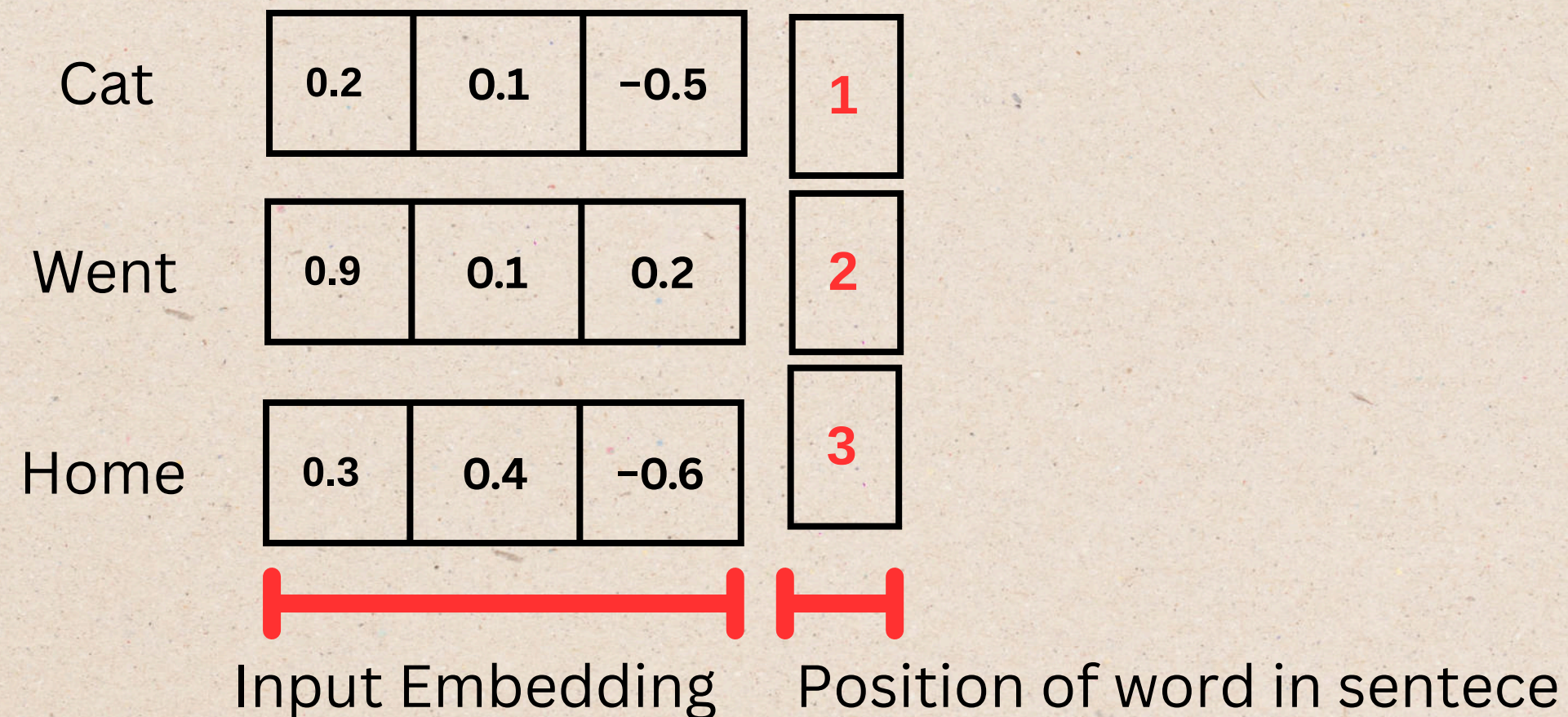
- Cat, Dog points to similar direction, while car..
- Why?
- Cat and Dog appears in similar context in training data



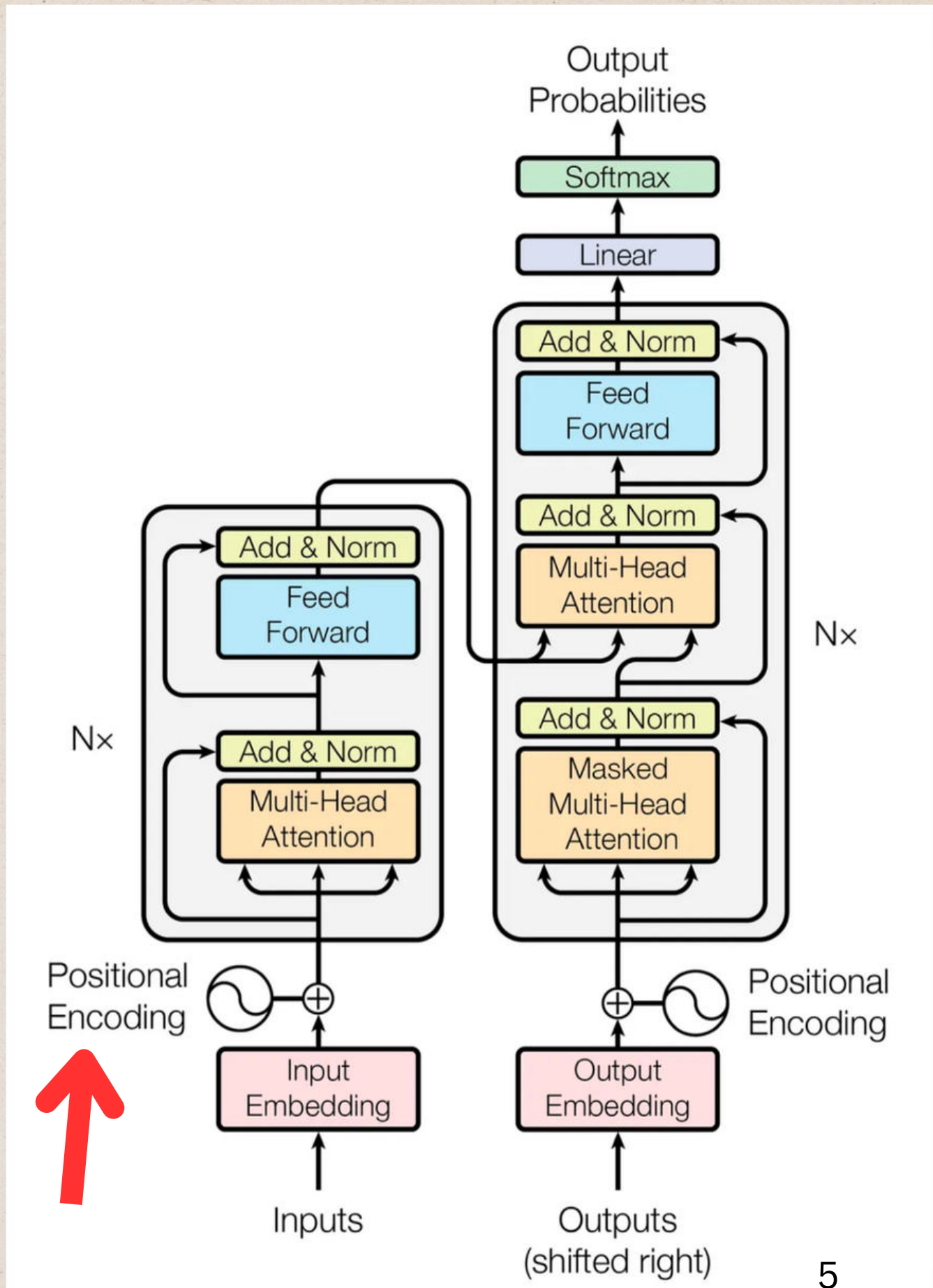
- $\text{King} - \text{Queen} = \text{Man} - \text{Woman}$
- $\text{Queen} = \text{King} - \text{Man} + \text{Woman}$
- spatial distance between words captures specific relationships

Positional Encoding

To know the position of the word..
How do we assign Positional Encoding ?

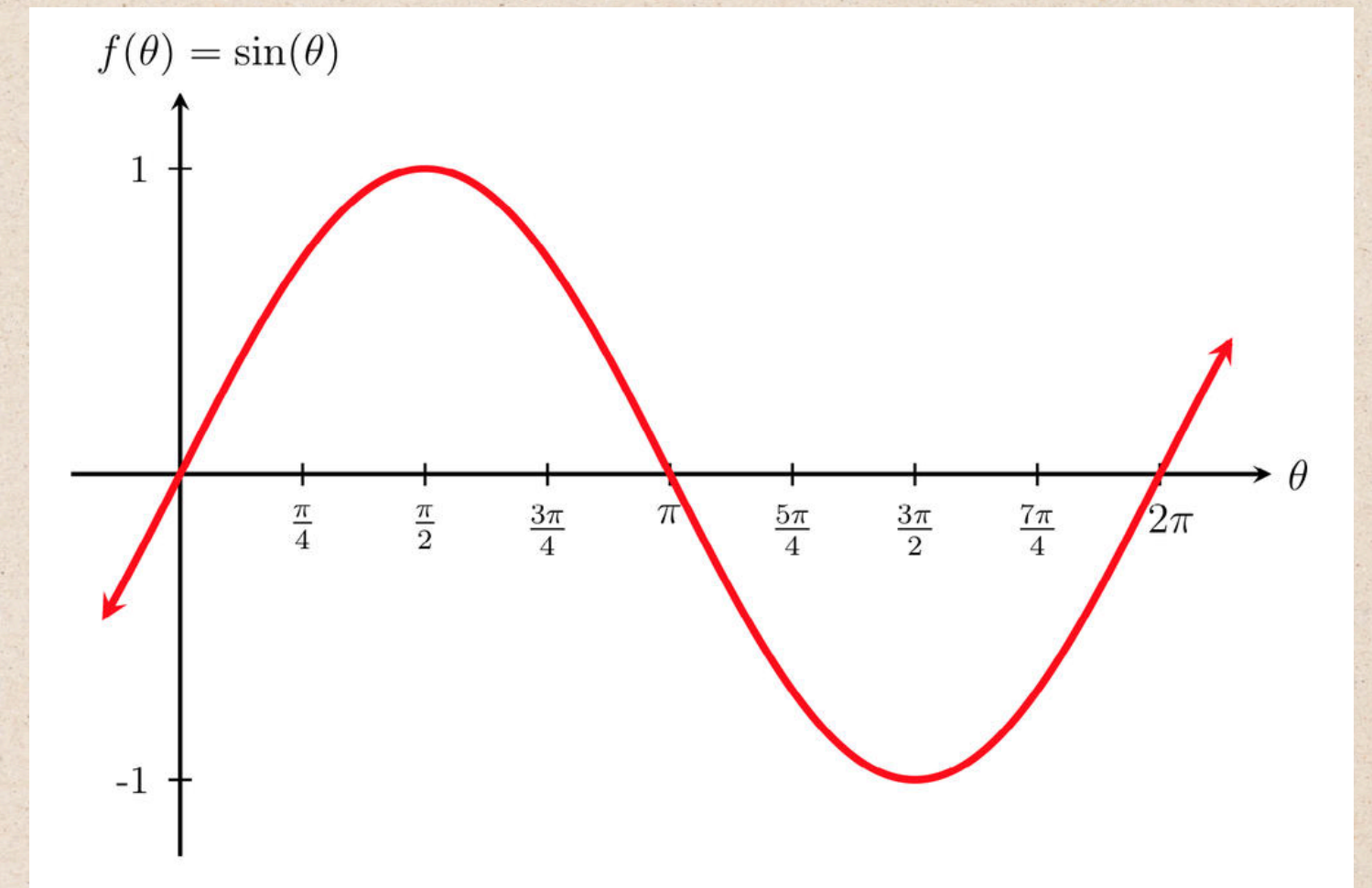
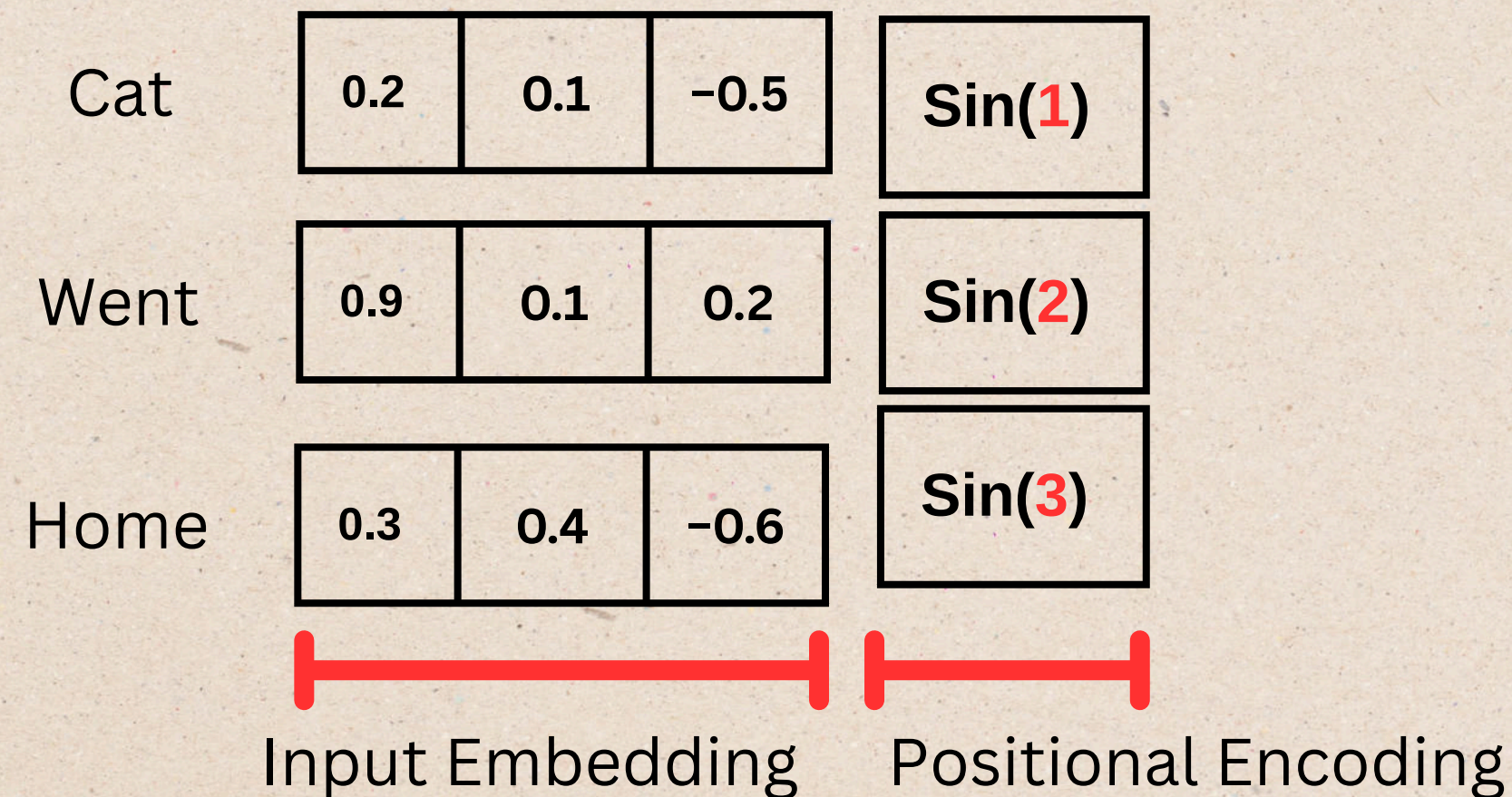


- We could assign 1 for first, 2 for second position & so on..
- But for large input, positional value becomes too large



Positional Encoding

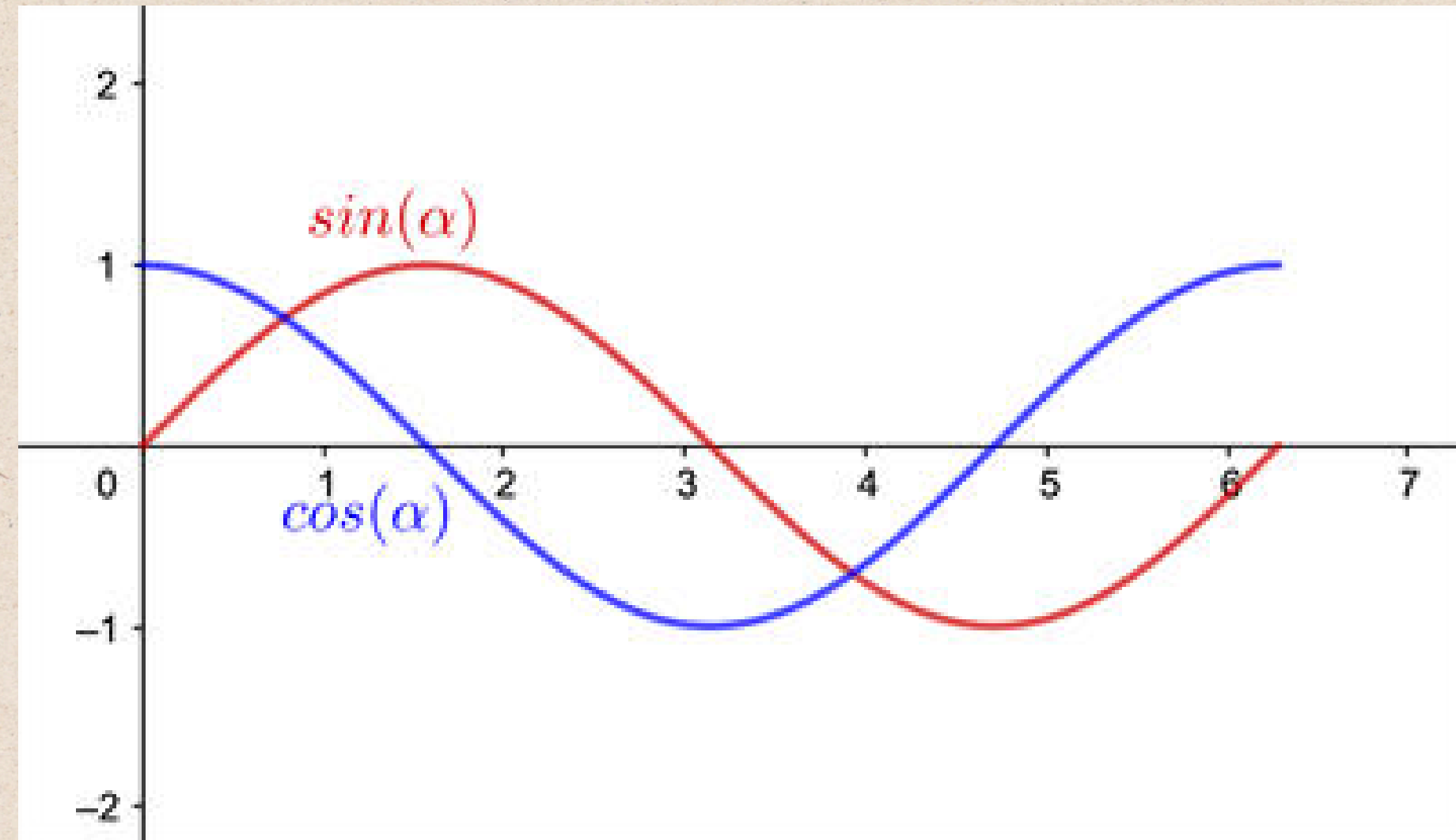
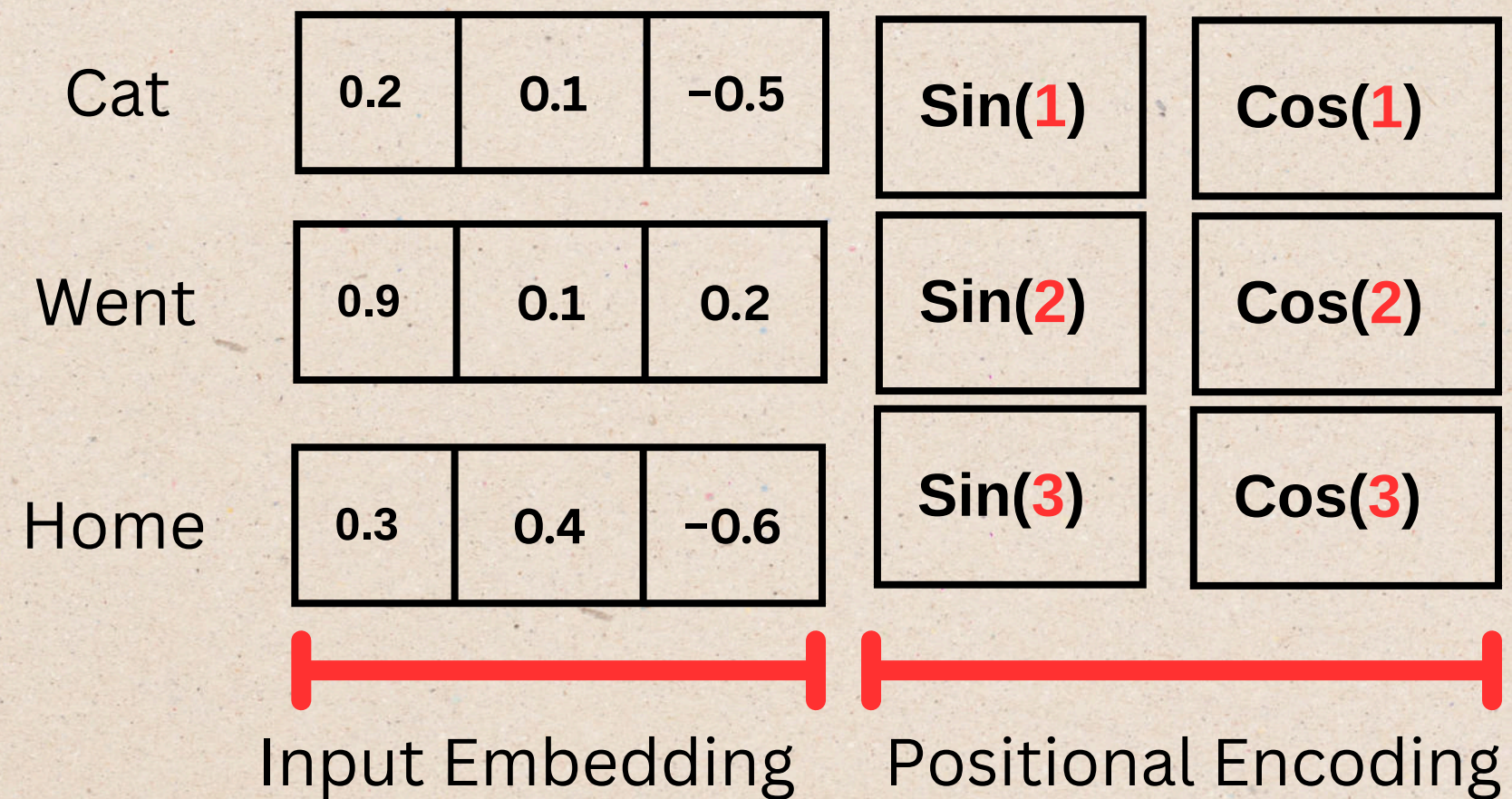
- We could use continuous function like sin function to prevent unbounded large value



- But 2 or more words could have a same Positional Encoding value

Positional Encoding

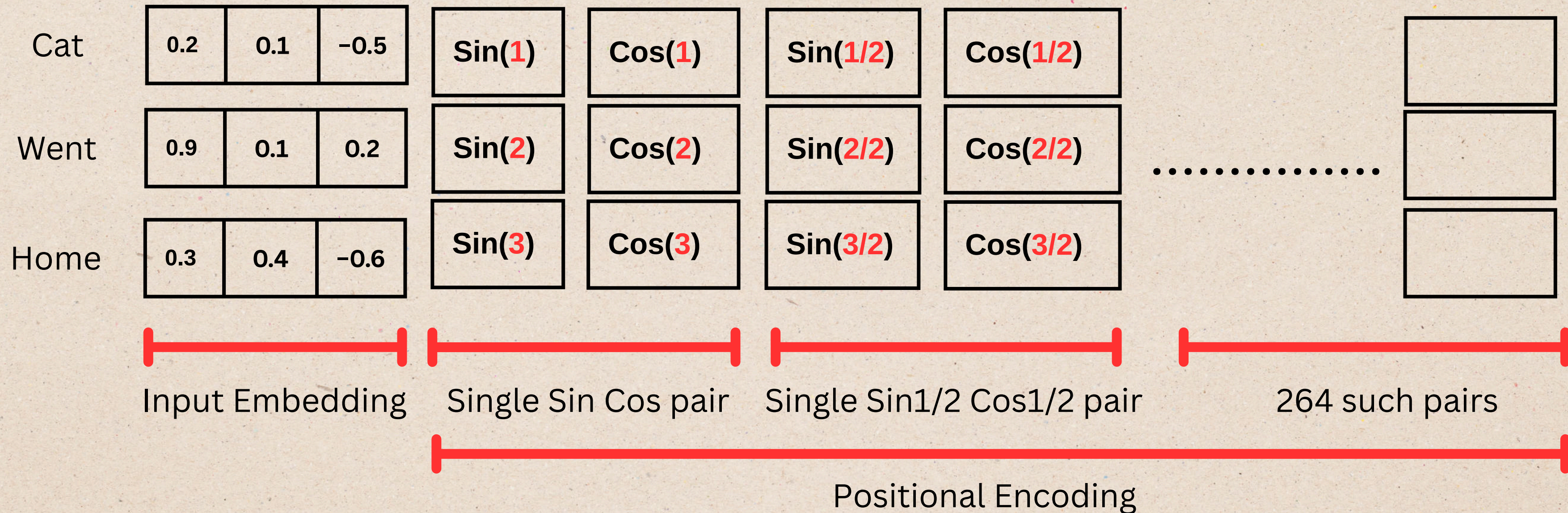
- We could use a pair of sin and cos function



- But, this pair would also not be sufficient

Positional Encoding

- So, we add Sin1/2 and Cos1/2 (half) functions



- Attention is all you need paper uses 264 such pairs of sine and cosine functions
- With total 512 dimensions for positional Encoding

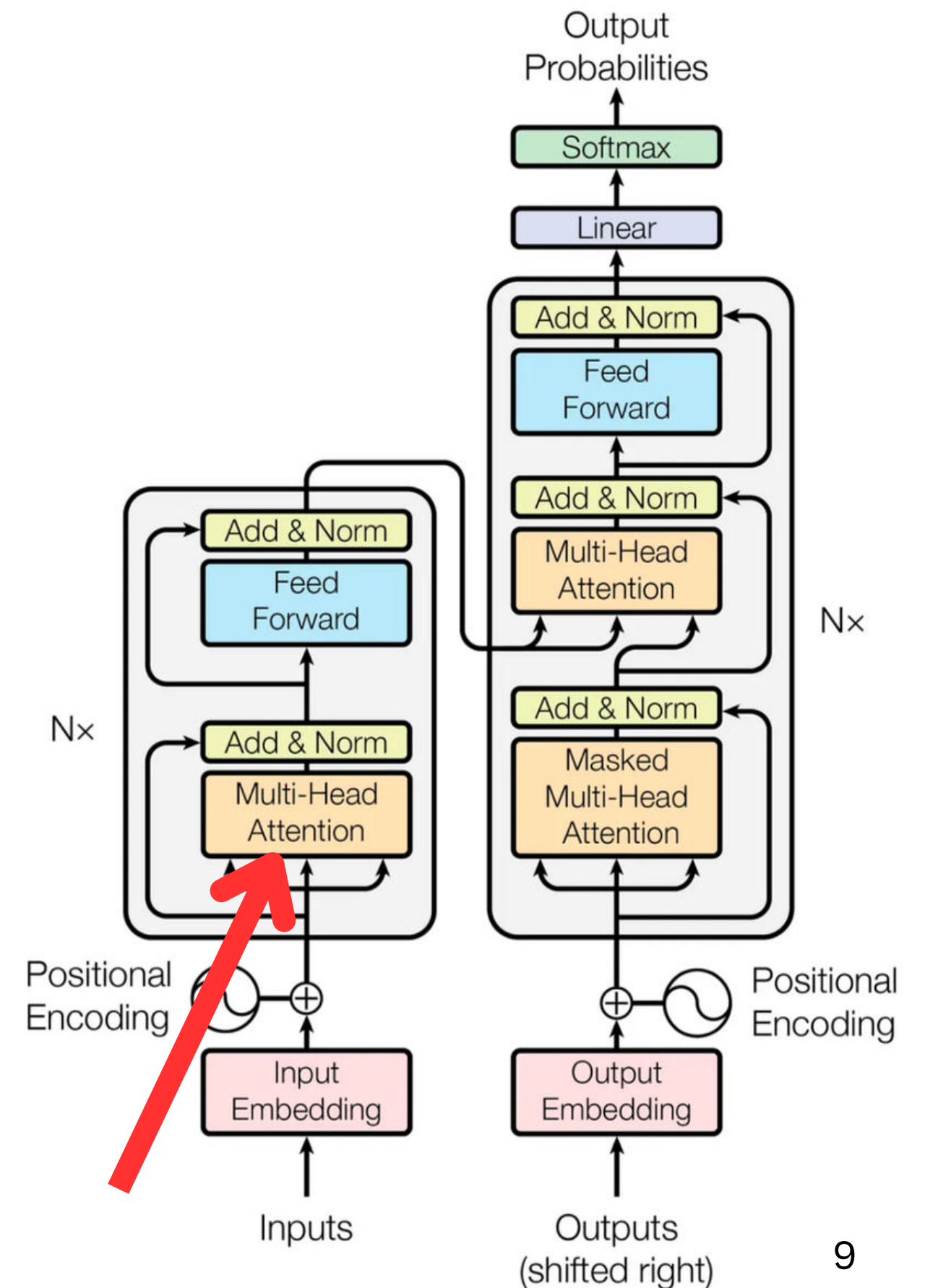
Self Attention

"Which words are important to pay attention to ?"

- Input Embedding + Positional Encoding goes to Attention Mechanism
- Each word creates 3 version of itself:
 - Query (Q) - What it's looking for in other words
 - Key (K) - What it's offering for other words
 - Value (V) - Actual information we want to extract

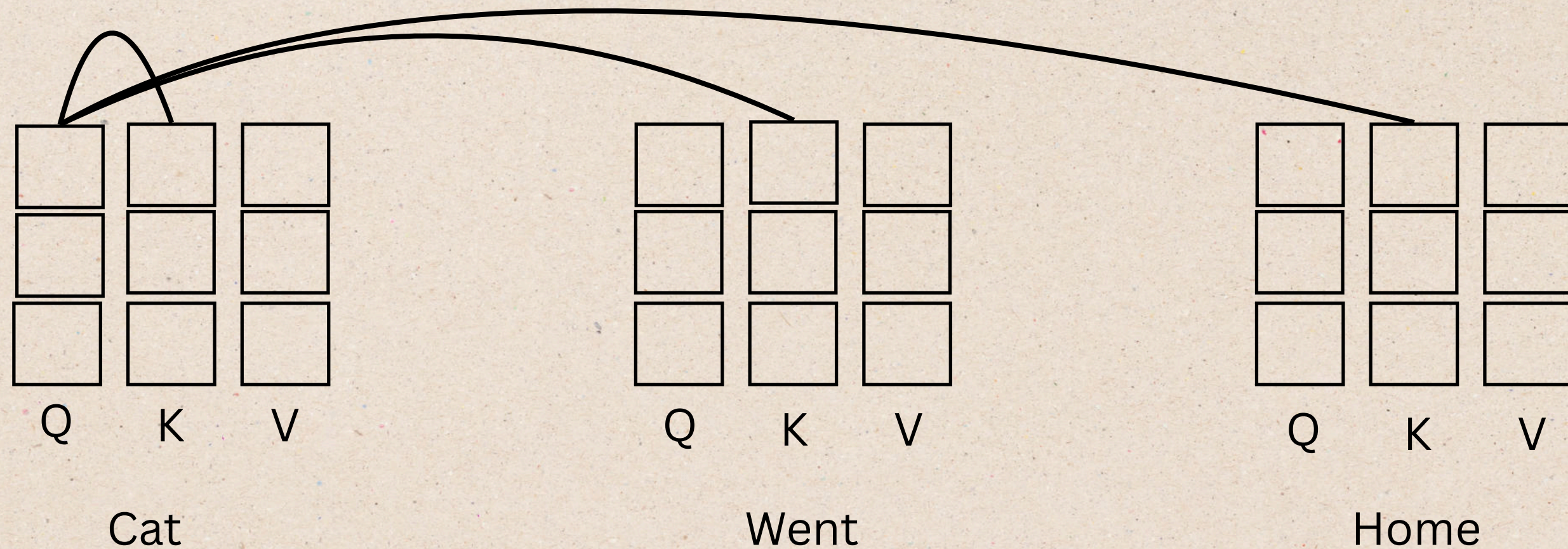
$$X = \text{Token Embedding} + \text{Positional Encoding}$$

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$



- W_Q, W_K, W_V are weights for Q, K, V

Self Attention



- We take **QUERY** of 1st token then do the dot product with **Key** of same token and others.

• We Get $Q.K^T$ as a result of dot product

$$\begin{array}{c} \square \\ \square \\ \square \end{array} \times \begin{array}{ccc} \square & \square & \square \end{array} = Q.K^T$$

K^T

Q

Self Attention

$$\begin{bmatrix} \square \\ \square \\ \square \end{bmatrix} \times \begin{bmatrix} \square & \square & \square \end{bmatrix}$$

K^T

$= Q.K^T$



$$\frac{Q.K^T}{\sqrt{d_k}}$$

$$\text{Softmax} \left(\begin{array}{c} \frac{Q.K^T}{\sqrt{d_k}} \\ \frac{Q.K^T}{\sqrt{d_k}} \\ \frac{Q.K^T}{\sqrt{d_k}} \end{array} \right)$$

Cat Went Home

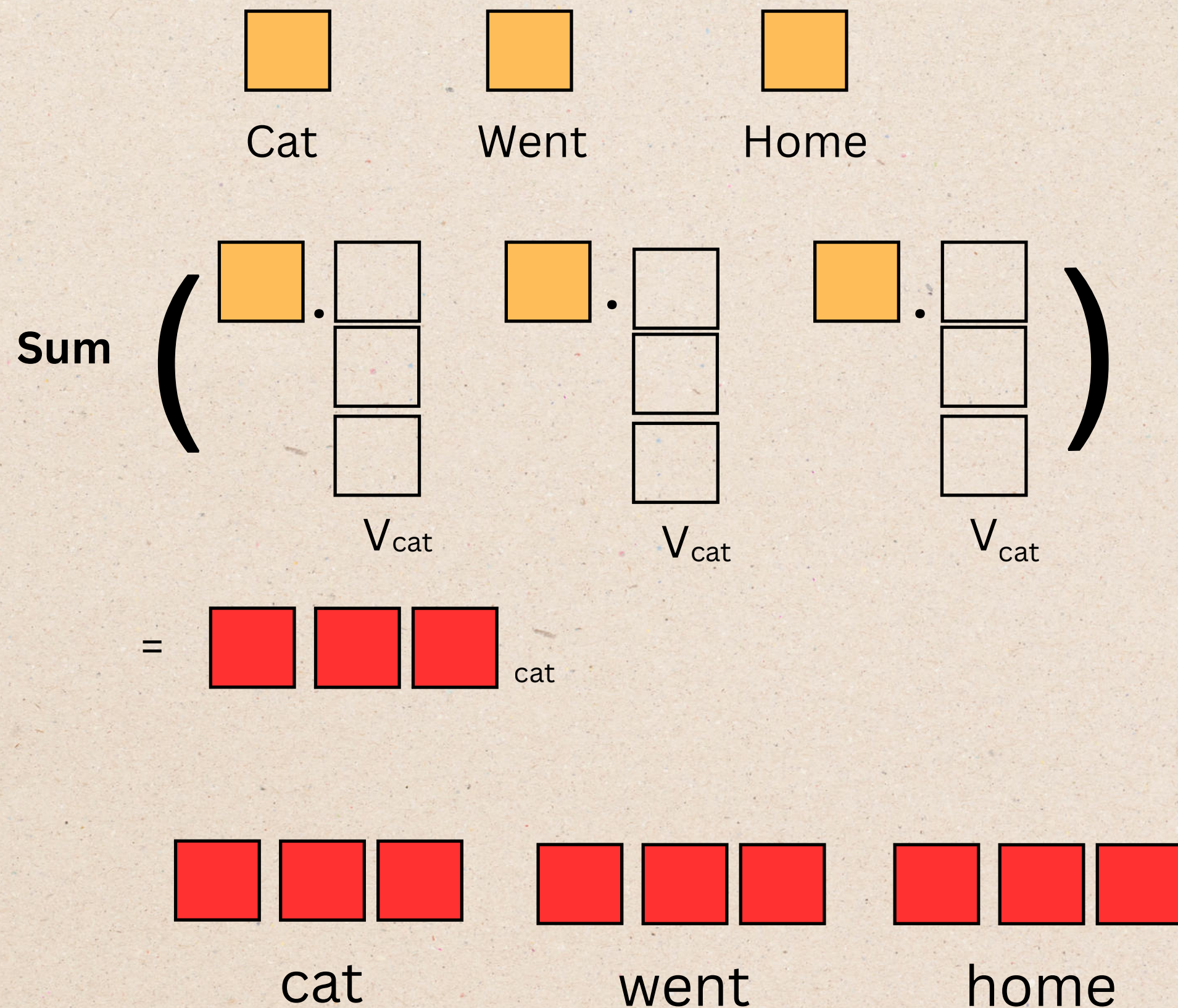
$=$

(Resulting Probabilities for each words)

- Then we divide this with $\sqrt{d_k}$ (the square root of the key dimension)
- To Prevent large values that destabilize training

- Then we put these values in Softmax function to convert values into Probabilities (attention score)
- Values between 0 and 1

Self Attention



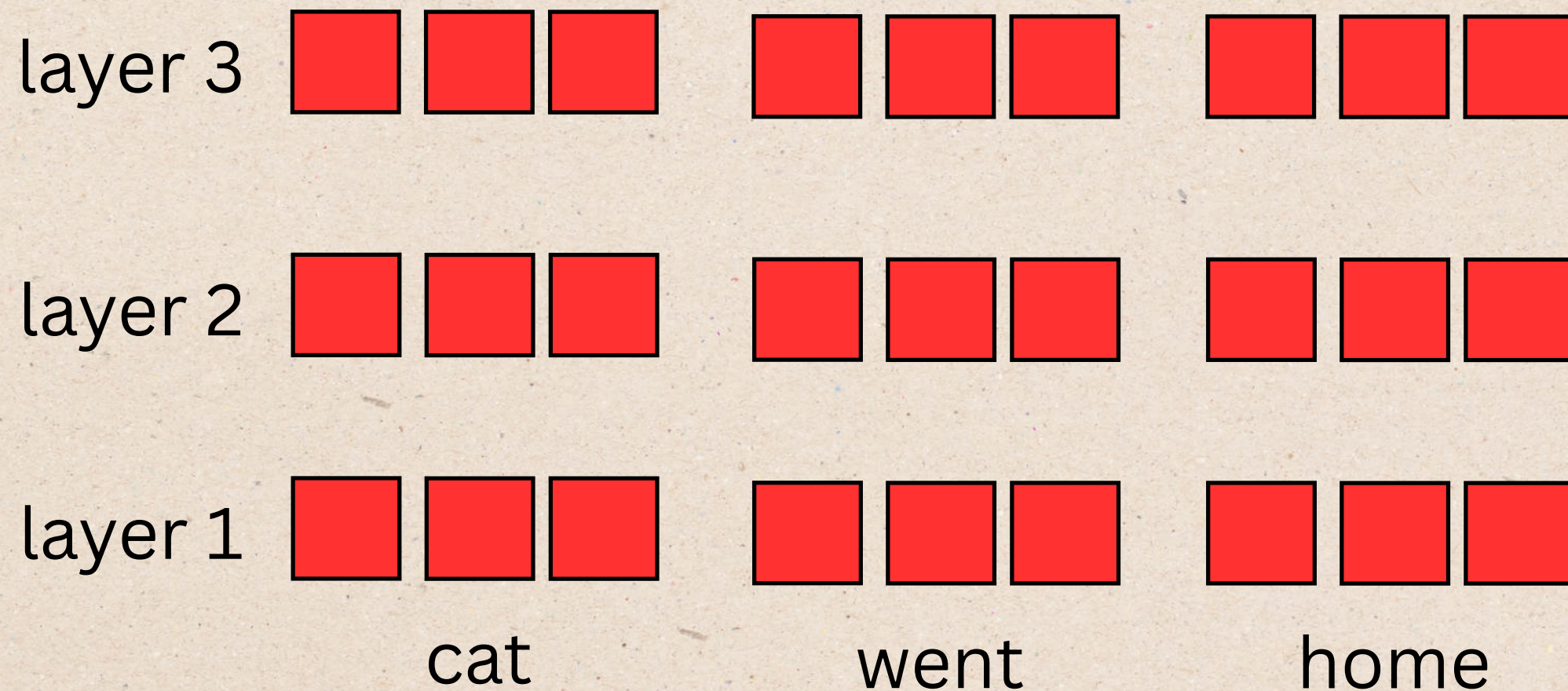
- Now we **multiply** these attention scores with Value (V) from earlier

$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

- Then, we **Sum** them.. (considering the value (v) came from "cat" word)
- This gives us attention score of the word (cat)

- Attention Score of all words

Multi -Head Attention



- Now we have attention score for all words.
- Same thing is done in Multiple Attention Layers
- Then all these layers are concatenated and multiplied with Weights

$$\text{MultiHead}(Q, K, V) = \text{Concat}(H_1, \dots, H_h)W_O$$

Add & Norm

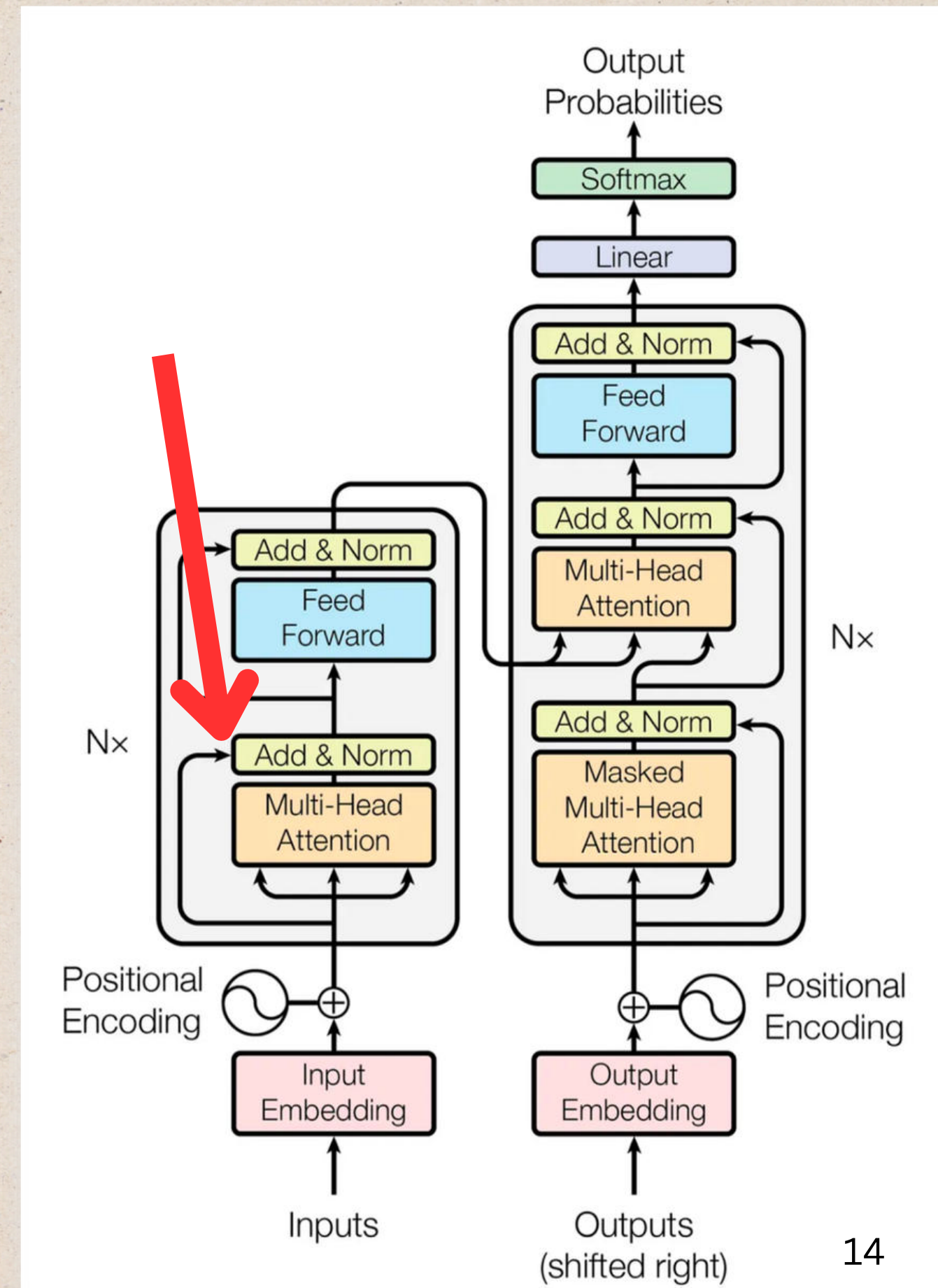
- Multi Head Attention Produces

$$\text{MHA}(X)$$

- Then, this output is combined with original input X (to multi-head attention)
- Using the residual connection $X + \text{MHA}(X)$
- Then, the normalization layer is applied

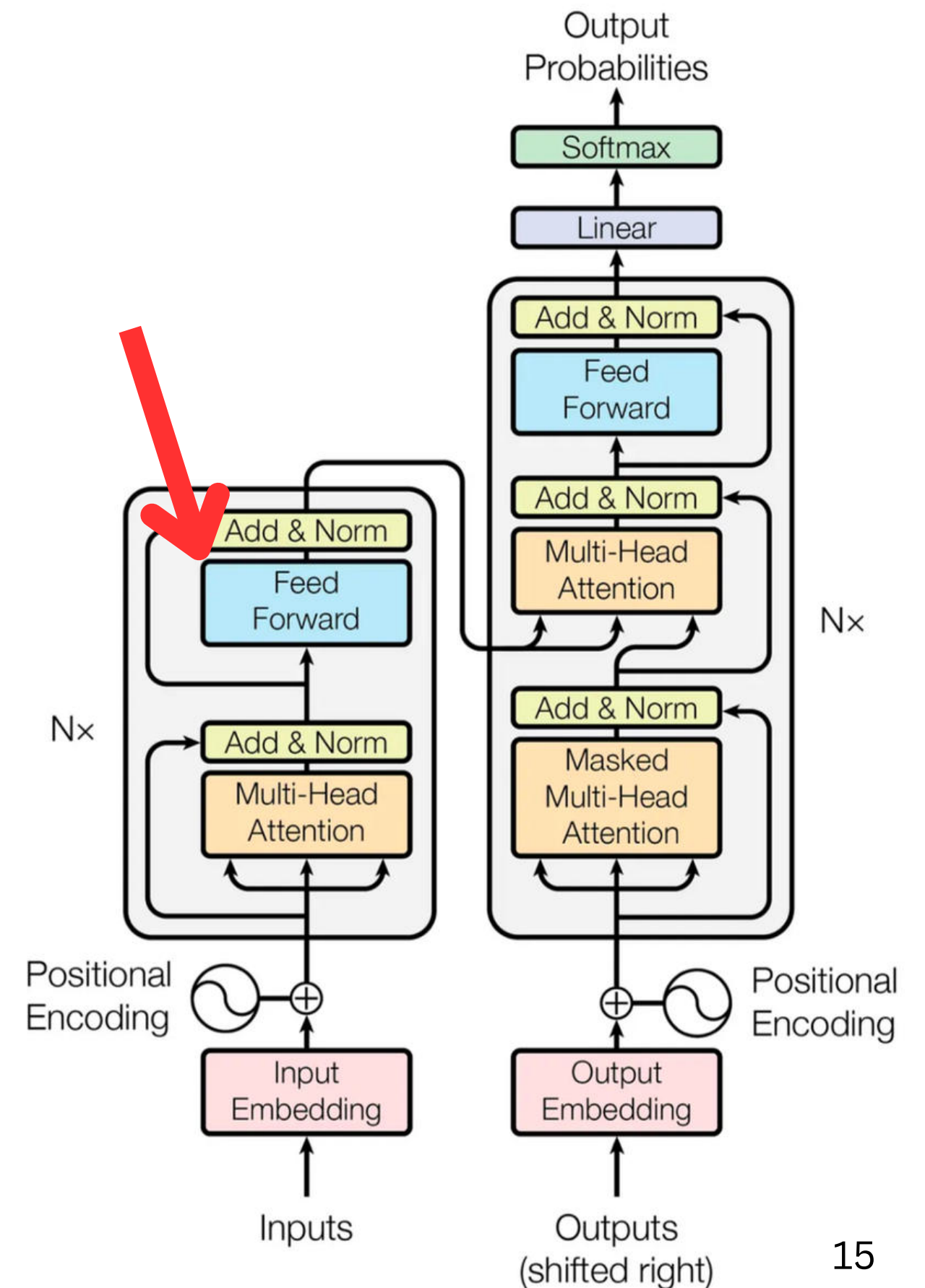
$$\text{LayerNorm}(X + \text{MHA}(X))$$

- We Normalize the values to improve training speed and stability.
- Neural Networks are unstable if numbers are too high or too low.



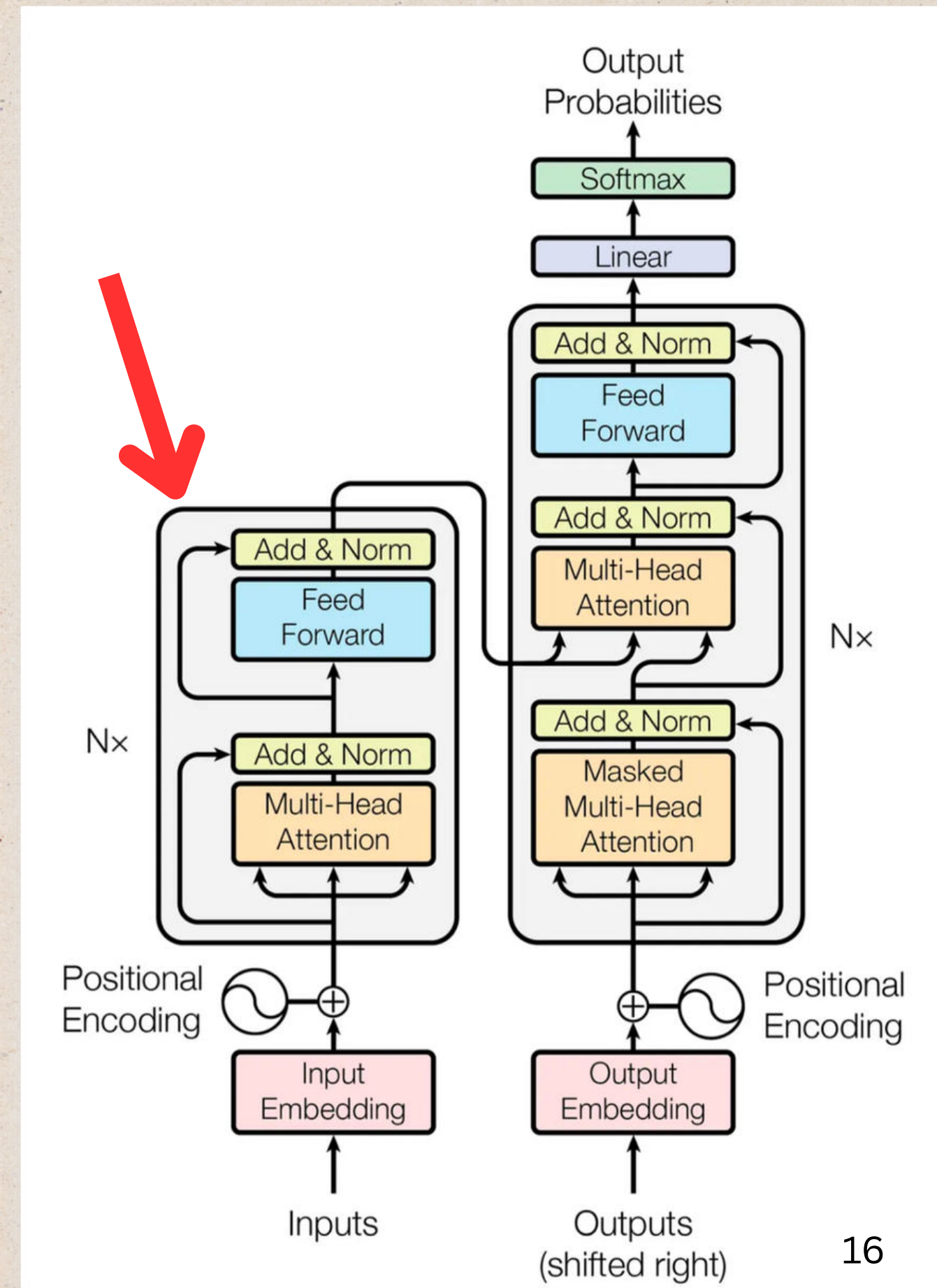
Feed Forward Network

- Each token independently passes through a small neural network.
- Usually has:
 - Linear Layer
 - Activation Functions (ReLU / GELU)
 - Another Linear Layer
- Adds nonlinearity
- Learns useful patterns



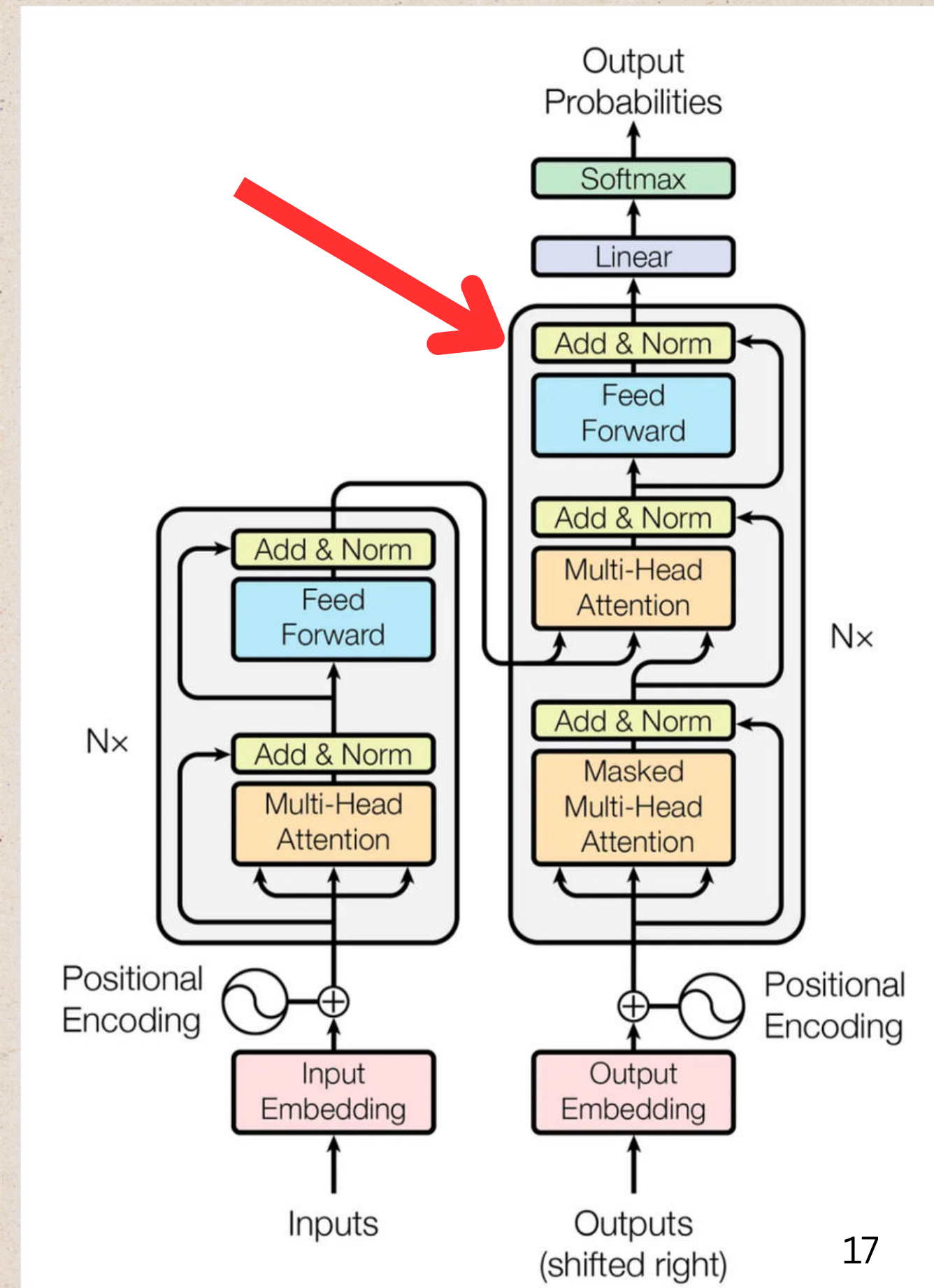
Encoder Block

- One Encoder Layer contains:
 - Multi-Head Attention
 - Add & Norm
 - Feed Forward
 - Add & Norm
- The Encoder Stack has multiple such encoder layers
- Encoder understands input
- has Bi-directional attention
- Used for Embeddings / Classification
- Outputs: Context aware representation
- Example: BERT



Decoder Block

- One Decoder Layer contains:
 - Masked Self-Attention
 - Prevents seeing future words
 - Encoder-Decoder Attention
 - Looks at encoder output
 - Feed Forward
 - Used for text generation
- The Decoder Stack has multiple such decoder layers
- Decoder generates output
- has masked attention
- Outputs: Used for text generation
- Example: GPT



Thank You